COMPETITION MASTER 01 — Raspberry Pi 5 Complete Smart Farm

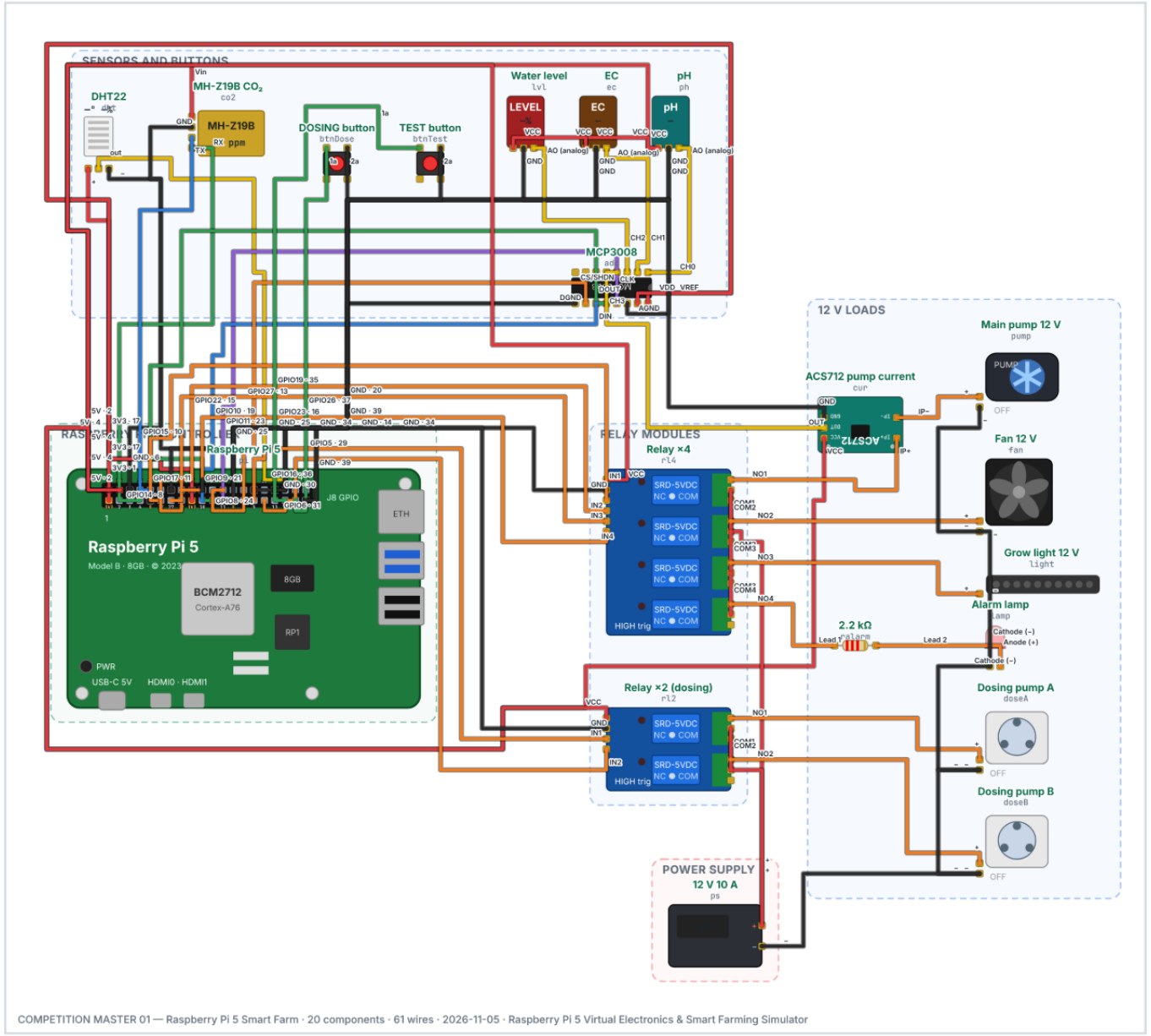
Complete study file

Read six sensors, run the AUTO / DOSING / TEST modes with protections and alarms, and show everything in the terminal and on a small Flask web page.

Controller	Raspberry Pi 5 Model B
Language	Python 3 (gpiozero, pyserial, adafruit_dht, Flask)
Program file	main.py
Status display	Terminal (STATUS lines) + Web tab (Flask page on port 5000).
Parts / wires	20 / 61
Exported	2026-11-05

DHT22 (GPIO16), MH-Z19B (UART0), MCP3008 (SPI0: pH, EC, level, ACS712), 4-channel + 2-channel relays, 12 V loads, two buttons.

1. Circuit diagram



20 parts · 61 wires

2. Wiring table — every wire explained

#	From	To	Type	Why	If disconnected
1	Raspberry Pi 5 · 3V3	DHT22 · +	Power	power for the DHT22	the DHT22 loses power — no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
2	Raspberry Pi 5 · GPIO16	DHT22 · out	Digital data	DHT22: one-wire digital data line to the Pi	no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
3	DHT22 · –	Raspberry Pi 5 · GND	Ground	ground (return path) of the DHT22	the DHT22 loses power — no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
4	Raspberry Pi 5 · 5V	MH-Z19B CO_2 · Vin	Power	power for the MH-Z19B CO_2 sensor	the MH-Z19B CO_2 sensor loses power — no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
5	MH-Z19B CO_2 · GND	Raspberry Pi 5 · GND	Ground	ground (return path) of the MH-Z19B CO_2 sensor	the MH-Z19B CO_2 sensor loses power — no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
6	Raspberry Pi 5 · GPIO14	MH-Z19B CO_2 · RX	Digital data	MH-Z19B CO_2 sensor: UART: sensor TX → controller RX, sensor RX ← controller TX (crossed)	no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
7	MH-Z19B CO_2 · TX	Raspberry Pi 5 · GPIO15	Digital data	MH-Z19B CO_2 sensor: UART: sensor TX → controller RX, sensor RX ← controller TX (crossed)	no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
8	Raspberry Pi 5 · 3V3	MCP3008 · VDD	Power	power for the MCP3008 ADC	the MCP3008 ADC loses power — all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
9	Raspberry Pi 5 · 3V3	MCP3008 · VREF	Power	power for the MCP3008 ADC	the MCP3008 ADC loses power — all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
10	MCP3008 · AGND	Raspberry Pi 5 · GND	Ground	ground (return path) of the MCP3008 ADC	the MCP3008 ADC loses power — all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
11	MCP3008 · DGND	Raspberry Pi 5 · GND	Ground	ground (return path) of the MCP3008 ADC	the MCP3008 ADC loses power — all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
12	Raspberry Pi 5 · GPIO11	MCP3008 · CLK	Digital data	MCP3008 ADC: SPI bus: the Pi has no analog inputs, the MCP3008 measures the four analog sensors	all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
13	MCP3008 · DOUT	Raspberry Pi 5 · GPIO9	Digital data	MCP3008 ADC: SPI bus: the Pi has no analog inputs, the MCP3008 measures the four analog sensors	all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
14	Raspberry Pi 5 · GPIO10	MCP3008 · DIN	Digital data	MCP3008 ADC: SPI bus: the Pi has no analog inputs, the MCP3008 measures the four analog sensors	all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
15	Raspberry Pi 5 · GPIO8	MCP3008 · CS/SHDN	Digital data	MCP3008 ADC: SPI bus: the Pi has no analog inputs, the MCP3008 measures the four analog sensors	all four analog readings become wrong (pH, EC, level, current): LOW WATER / SENSOR ERROR, irrigation blocked
16	Raspberry Pi 5 · 5V	pH · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
17	pH · GND	Raspberry Pi 5 · GND	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR

#	From	To	Type	Why	If disconnected
18	pH · VCC	EC · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
19	EC · GND	pH · GND	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
20	EC · VCC	Water level · VCC	Power	power for the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
21	Water level · GND	EC · GND	Ground	ground (return path) of the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
22	pH · AO (analog)	MCP3008 · CH0	Analog signal	pH sensor module: analog voltage: pH = 3.5 × V	pH reads wrong (near 0 → below 1): SENSOR ERROR
23	EC · AO (analog)	MCP3008 · CH1	Analog signal	EC sensor module: analog voltage: EC = V ÷ 0.5	EC reads wrong (near 0): OUT OF RANGE
24	Water level · AO (analog)	MCP3008 · CH2	Analog signal	water-level sensor: analog voltage: 3 V = 100 %	level reads wrong (near 0 %): LOW WATER, irrigation and dosing refused
25	Raspberry Pi 5 · 5V	ACS712 pump current · VCC	Power	power for the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE
26	ACS712 pump current · GND	Raspberry Pi 5 · GND	Ground	ground (return path) of the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE
27	ACS712 pump current · OUT	MCP3008 · CH3	Analog signal	ACS712 current sensor: analog voltage: 2.5 V + 0.185 V per ampere of pump current	the pump current reads wrong: the next irrigation reports PUMP FAILURE
28	Raspberry Pi 5 · 5V	Relay ×4 · VCC	Power	power for the 4-channel relay module	the relay coils lose power: none of pump, fan, light, alarm lamp can run
29	Relay ×4 · GND	Raspberry Pi 5 · GND	Ground	ground (return path) of the 4-channel relay module	the relay coils lose power: none of pump, fan, light, alarm lamp can run
30	Raspberry Pi 5 · GPIO17	Relay ×4 · IN1	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
31	Raspberry Pi 5 · GPIO22	Relay ×4 · IN2	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
32	Raspberry Pi 5 · GPIO27	Relay ×4 · IN3	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
33	Raspberry Pi 5 · GPIO23	Relay ×4 · IN4	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
34	Raspberry Pi 5 · 5V	Relay ×2 (dosing) · VCC	Power	power for the 2-channel relay module (dosing)	both dosing relays lose power: no dosing at all
35	Relay ×2 (dosing) · GND	Raspberry Pi 5 · GND	Ground	ground (return path) of the 2-channel relay module (dosing)	both dosing relays lose power: no dosing at all
36	Raspberry Pi 5 · GPIO5	Relay ×2 (dosing) · IN1	Digital data	2-channel relay module (dosing): relay input: a HIGH level closes that channel	that dosing pump never runs although the program prints DOSING …
37	Raspberry Pi 5 · GPIO6	Relay ×2 (dosing) · IN2	Digital data	2-channel relay module (dosing): relay input: a HIGH level closes that channel	that dosing pump never runs although the program prints DOSING …
38	ACS712 pump current · IP−	Main pump 12 V · +	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply −	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
39	2.2 kΩ · Lead 2	Alarm lamp · Anode (+)	Digital data	alarm lamp: 12 V lamp (LED + 2.2 kΩ)	alarms are still printed but the lamp never lights
40	12 V 10 A · +	Relay ×4 · COM1	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply −	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current

#	From	To	Type	Why	If disconnected
41	Relay ×4 · NO1	ACS712 pump current · IP+	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply −	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
42	Relay ×4 · COM1	Relay ×4 · COM2	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply −	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
43	Relay ×4 · NO2	Fan 12 V · +	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply −	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
44	Relay ×4 · COM2	Relay ×4 · COM3	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply −	the light stays off although the program prints light=ON
45	Relay ×4 · NO3	Grow light 12 V · +	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply −	the light stays off although the program prints light=ON
46	Relay ×4 · COM3	Relay ×4 · COM4	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply −	alarms are still printed but the lamp never lights
47	Relay ×4 · NO4	2.2 kΩ · Lead 1	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply −	alarms are still printed but the lamp never lights
48	12 V 10 A · +	Relay ×2 (dosing) · COM1	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
49	Relay ×2 (dosing) · NO1	Dosing pump A · +	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
50	Relay ×2 (dosing) · COM1	Relay ×2 (dosing) · COM2	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply −	dose B is not delivered although DOSING B is printed
51	Relay ×2 (dosing) · NO2	Dosing pump B · +	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply −	dose B is not delivered although DOSING B is printed
52	Main pump 12 V · −	Fan 12 V · −	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply −	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
53	Fan 12 V · −	Grow light 12 V · −	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply −	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
54	Grow light 12 V · −	Alarm lamp · Cathode (−)	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply −	the light stays off although the program prints light=ON
55	Alarm lamp · Cathode (−)	Dosing pump A · −	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
56	Dosing pump A · −	Dosing pump B · −	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
57	Dosing pump B · −	12 V 10 A · −	Ground	ground (return path) of the dosing pump B	the dosing pump B loses power — dose B is not delivered although DOSING B is printed
58	Raspberry Pi 5 · GPIO26	DOSING button · 1a	Digital data	DOSING button: input with internal pull-up: pressed = connected to GND = 0	the button does nothing (the pull-up keeps the input at "not pressed")
59	DOSING button · 2a	Raspberry Pi 5 · GND	Ground	ground (return path) of the DOSING button	the DOSING button loses power — the button does nothing (the pull-up keeps the input at "not pressed")
60	Raspberry Pi 5 · GPIO19	TEST button · 1a	Digital data	TEST button: input with internal pull-up: pressed = connected to GND = 0	the button does nothing (the pull-up keeps the input at "not pressed")
61	TEST button · 2a	Raspberry Pi 5 · GND	Ground	ground (return path) of the TEST button	the TEST button loses power — the button does nothing (the pull-up keeps the input at "not pressed")

3. Parts list

#	Component	Model / reference	Qty	Key properties	Voltage	Simulation level	IDs	Assembly notes
1	Raspberry Pi 5 Model B (8GB)	Raspberry Pi 5 Model B 8GB · Raspberry Pi Ltd	1	1-Wire (dtoverlay=w1-gpio): no; UART0 enabled (dtparam=uart0=on): yes	5 V USB-C in, 3.3 V GPIO logic	Fully Simulated	pi	
2	DHT22 temperature/humidity	DHT22 / AM2302 module · Aosong	1	Module with pull-up resistor: yes	3.3–6 V	Fully Simulated	dht	
3	MH-Z19B CO_2 sensor	MH-Z19B · Winsen	1	—	4.5–5.5 V supply, 3.3 V UART	Fully Simulated	co2	
4	MCP3008 8-channel 10-bit ADC	MCP3008-I/P · Microchip	1	—	2.7–5.5 V	Fully Simulated	adc	
5	pH sensor (analog)	Analog pH probe + board (SEN0161-style)	1	—	3.3–5 V	Functional Model	ph	
6	EC conductivity sensor (analog)	Analog EC probe + board	1	—	3.3–5 V	Functional Model	ec	
7	Water level sensor (analog)	Analog water-level sensor	1	—	3.3–5 V	Functional Model	lvl	
8	ACS712 current sensor (5 A)	ACS712ELCTR-05B · Allegro	1	Version: 5 A (185 mV/A)	5 V supply	Fully Simulated	cur	
9	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 4; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	Fully Simulated	rl4	
10	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 2; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	Fully Simulated	rl2	
11	DC power supply	Bench / adapter DC supply	1	Voltage: 12 V; Current limit: 10 A	—	Fully Simulated	ps	
12	Water pump (12 V DC)	12 V DC pump	1	Rated power: 6 W	12 V	Functional Model	pump	
13	Cooling fan (DC)	12 V DC fan	1	Rated power: 2.4 W	12 V	Functional Model	fan	
14	LED grow light	12 V LED grow bar	1	Rated power: 36 W	12 V	Functional Model	light	
15	Resistor	¼ W carbon-film resistor	1	Resistance: 2.2 kΩ	—	Fully Simulated	ralarm	
16	LED	5 mm LED	1	Colour: red	Vf ≈ 1.9–2.9 V, 20 mA max	Fully Simulated	lamp	
17	Peristaltic dosing pump	12 V peristaltic pump	2	Rated power: 3.6 W	12 V	Functional Model	doseA, doseB	
18	Push button	6×6 mm tactile switch	2	—	—	Fully Simulated	btnDose, btnTest	
19	Jumper wires	—	61	16× red, 5× yellow, 18× black, 4× green, 2× blue, 1× purple, 15× orange	—	—		Use male–female jumpers for the Pi header; keep power wires short.

4. Settings (constants at the top of the program)

Name	Value	Meaning
MAX_TEMP	30	فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP	27	تحت 27 درجة: إيقاف المروحة
MIN_WATER	20	أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS	5	مدة الري بالثواني
IRRIGATION_HOURS	[7, 12, 17]	ساعات الري في اليوم
LIGHT_ON_HOUR	6	ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR	20	ساعة إطفاء الإضاءة
DOSE_A_ML	10	جرعة السماد A بالمليلتر
DOSE_B_ML	5	جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC	2	تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN	5.5	أقل pH مقبول
PH_MAX	6.5	أعلى pH مقبول
EC_MIN	1.0	أقل EC مقبول (mS/cm)
EC_MAX	2.5	أعلى EC مقبول
MIN_PUMP_AMPS	0.2	أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS	2	مدة تشغيل كل جهاز في وضع الاختبار

Training values you may change; they are not certified settings for every crop. Dose accuracy depends on the calibration of the real pumps.

5. The program in 9 sections

1 · Setup and wiring

The settings first, then one line per device: every sensor and every relay gets a name and its GPIO / ADC channel. The names are what the rest of the program uses.

- Change a threshold here and the whole program follows it.
- GPIO numbers are BCM numbers (GPIO17 = physical pin 11).

main.py

```
# ---------- الإعدادات: قيم تدريبية يمكنك تغييرها (1 ----------
# تشغيل المروحة: فوق 30 درجة
MAX_TEMP = 30
# إيقاف المروحة: تحت 27 درجة
FAN_OFF_TEMP = 27
# لتشغيل المضخة (%) أقل مستوى ماء مسموح
MIN_WATER = 20
# مدة الري بالثواني
PUMP_SECONDS = 5
# ساعات الري في اليوم
IRRIGATION_HOURS = [7, 12, 17]
# ساعة تشغيل الإضاءة
LIGHT_ON_HOUR = 6
# ساعة إطفاء الإضاءة
LIGHT_OFF_HOUR = 20
# جرعة السماد A بالمليلتر
DOSE_A_ML = 10
# جرعة السماد B بالمليلتر
DOSE_B_ML = 5
# تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
FLOW_ML_PER_SEC = 2
# أقل pH مقبول
PH_MIN = 5.5
# أعلى pH مقبول
PH_MAX = 6.5
# أقل EC مقبول (mS/cm)
EC_MIN = 1.0
# أعلى EC مقبول
EC_MAX = 2.5
# أعلى CO2 مقبول (ppm)
CO2_MAX = 1500
# أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
MIN_PUMP_AMPS = 0.2
# مدة تشغيل كل جهاز في وضع الاختبار
TEST_SECONDS = 2


# >>> PINS
# ---------- التوصيلات: كل جهاز على رجله (2 ----------
# البيانات على GPIO16
dht = adafruit_dht.DHT22(board.D16)
# UART0 على co2_port = serial.Serial("/dev/serial0", 9600, timeout=1)  # MH-Z19B (GPIO15 و GPIO14)
# حساس pH على قناة ADC رقم 0
ph_adc = MCP3008(channel=0)
# حساس EC على القناة 1
ec_adc = MCP3008(channel=1)
# حساس مستوى الماء على القناة 2
level_adc = MCP3008(channel=2)
```

```
حساس تيار المضخة ACS712 على القناة 3 #                    current_adc = MCP3008(channel=3)  37
مرحّل مضخة الري على GPIO17 #                              pump = OutputDevice(17)  38
مرحّل المروحة على GPIO22 #                                fan = OutputDevice(22)  39
مرحّل الإضاءة على GPIO27 #                               light = OutputDevice(27)  40
مرحّل مصباح الإنذار على GPIO23 #                          alarm_lamp = OutputDevice(23)  41
مرحّل مضخة الجرعات A على GPIO5 #                               dose_a = OutputDevice(5)  42
مرحّل مضخة الجرعات B على GPIO6 #                               dose_b = OutputDevice(6)  43
زر وضع الجرعات على GPIO26 (إلى GND) #                      dosing_button = Button(26)  44
زر وضع الاختبار على GPIO19 (إلى GND) #                       test_button = Button(19)  45
 46  # <<< PINS
```

2 · Reading the sensors

One function reads everything and stores it in variables. A missing reading becomes None/null, so the alarms can see it. Analog sensors give volts; one formula per sensor turns volts into pH, EC, % and amperes.

- pH = 3.5 × V · EC = V ÷ 0.5 · level = V ÷ 3 × 100 · amps = (V − 2.5) ÷ 0.185 (the simulator modules; calibrate real probes).
- DHT22 needs 2 s between readings: the program reads it at most every 2 s.

main.py

```
قراءة CO2 من MH-Z19B عبر UART #                                    :()def read_co2  64
أمر طلب القراءة (9 بايت) #  co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))      65
ننتظر رداً من 9 بايت #                          reply = co2_port.read(9)      66
لا يوجد رد = الحساس مفصول #                                :if len(reply) != 9      67
تعني: لا توجد قراءة return None                                    # None          68
ppm التركيز = البايت 2 و 3 #                 return reply[2] * 256 + reply[3]      69
 72  # >>> SENSORS
قراءة كل الحساسات #                                    :()def read_sensors  73
سنغيّر هذه #  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time      74
↪ المتغيرات العامة
قراءة جديدة كل ثانيتين فقط :if time.time() - last_dht_time >= 2:            # DHT22      75
نحفظ وقت هذه القراءة #                 ()last_dht_time = time.time          76
نحاول القراءة #                                        :try          77
الحرارة C° #           temperature = dht.temperature              78
الرطوبة % #                humidity = dht.humidity              79
فشل القراءة (الحساس مفصول) #                          :except RuntimeError          80
لا توجد قراءة حرارة #                      temperature = None              81
ولا رطوبة #                          humidity = None              82
(أو None) co2 = read_co2()                                  # CO2 ppm      83
الجهد × ph = round(ph_adc.voltage * 3.5, 2)                # pH = 3.5      84
الجهد ÷ 0.5 = ec = round(ec_adc.voltage / 0.5, 2)                # EC      85
فولت %100 = water_level = round(level_adc.voltage / 3.0 * 100)  # 3      86
فولت صفر أمبير = pump_amps = round((current_adc.voltage - 2.5) / 0.185, 2)  # ACS712: 2.5      87
 88  # <<< SENSORS
```

3 · Driving the outputs

A GPIO cannot power a pump: it switches the input of a relay module, and the relay contact switches the 12 V load. on() closes the relay, off() opens it.

- Loads: pump, fan, light and dosing pumps on the 12 V supply, never on the controller pins.
- The alarm lamp is a 12 V lamp on its own relay channel.

main.py

```
مرحّل مضخة الري على GPIO17 #                              pump = OutputDevice(17)  38
مرحّل المروحة على GPIO22 #                                fan = OutputDevice(22)  39
مرحّل الإضاءة على GPIO27 #                               light = OutputDevice(27)  40
مرحّل مصباح الإنذار على GPIO23 #                          alarm_lamp = OutputDevice(23)  41
مرحّل مضخة الجرعات A على GPIO5 #                               dose_a = OutputDevice(5)  42
مرحّل مضخة الجرعات B على GPIO6 #                               dose_b = OutputDevice(6)  43
لعرض حالة جهاز "OFF" أو "def on_off(device):                                  # "ON  204
الجهاز يعمل الآن؟ #                                :if device.is_active      205
يعمل #                                  "return "ON          206
متوقف #                                     "return "OFF      207
```

4 · Conditions and protection

water_ok() is the one place that decides "is there enough water?". Irrigation checks it before and during every second, and checks the pump current: a pump that runs dry or is not connected draws no current, so the program

stops it and remembers the failure.

- A level above 100 % is treated as an invalid reading, not as "full".
- The pump is always switched off at the end, whatever stopped the irrigation.

main.py

```
def water_ok():  # 91
    return water_level >= MIN_WATER and water_level <= 100  # قراءة غير صالحة = أكثر من 100 % 92
# >>> IRRIGATION  99
def irrigate():  # 100
    global pump_failed  # قد نغيّر هذا المتغير العام 101
    print("IRRIGATION start")  # رسالة بداية الري 102
    for second in range(PUMP_SECONDS):  # ثانية بعد ثانية 103
        read_sensors()  # قراءة جديدة قبل كل ثانية 104
        if not water_ok():  # الماء نقص أثناء الري 105
            # print("IRRIGATION stopped: LOW WATER")  نخبر بالسبب 106
            break  # نخرج من الحلقة فوراً 107
        pump.on()  # تشغيل المضخة 108
        time.sleep(1)  # انتظار ثانية 109
        read_sensors()  # نقرأ تيار المضخة الآن 110
        if pump_amps < MIN_PUMP_AMPS:  # المضخة لا تسحب تياراً = لا تعمل 111
            pump_failed = True  # نتذكر العطل (يظهر الإنذار) 112
            print("PUMP FAILURE: no current")  # رسالة العطل 113
            break  # نتوقف عن الري 114
    pump.off()  # إيقاف المضخة دائماً في النهاية 115
    print("IRRIGATION end")  # رسالة نهاية الري 116
# <<< IRRIGATION 117
```

5 · AUTO mode

Every second: the fan follows the temperature with two thresholds (ON above 30 °C, OFF below 27 °C — between them it keeps its state, so it does not click on and off), the light follows the clock (06:00–20:00), and irrigation starts once at 07:00, 12:00 and 17:00 if there is water.

- No temperature reading → the fan runs (the safe side for the plants).
- last_irrigation_hour prevents watering again and again during the same hour.

main.py

```
def auto_control():  # 154
    global last_irrigation_hour  # سنغيّر هذا المتغير العام 155
    hour = time.localtime().tm_hour  # الساعة الحالية (0 إلى 23) 156
# >>> FAN 157
    if temperature is None:  # لا قراءة حرارة: نشغّل المروحة للأمان 158
        fan.on()  # تشغيل المروحة 159
    elif temperature > MAX_TEMP:  # حار جداً 160
        fan.on()  # تشغيل المروحة 161
    elif temperature < FAN_OFF_TEMP:  # برد بما يكفي 162
        fan.off()  # إيقاف المروحة (بين 27 و 30 تبقى كما هي) 163
# <<< FAN 164
# >>> LIGHT 165
    if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR:  # داخل ساعات الإضاءة 166
        light.on()  # تشغيل الإضاءة 167
    else:  # خارج ساعات الإضاءة 168
        light.off()  # إطفاء الإضاءة 169
# <<< LIGHT 170
    if hour in IRRIGATION_HOURS and hour != last_irrigation_hour:  # حان وقت ري جديد 171
        last_irrigation_hour = hour  # نتذكر الساعة حتى لا نكرر الري 172
        if water_ok():  # يوجد ماء؟ 173
            irrigate()  # نروي 174
        else:  # لا يوجد ماء 175
            print("IRRIGATION skipped: LOW WATER")  # نتخطى الري 176
def dosing():  # 177
    pump_a.off()  # نتأكد أن المضخات مطفأة 178
    pump_b.off()  # 179
    if not water_ok():  # لا نضخ في خزان فارغ 180
        print("DOSING skipped: LOW WATER")  # 181
        return  # 182
    print("DOSING start")  # 183
    pump_b.on()  # المضخة B أولاً 184
    time.sleep(dose_b_ml / DOSE_FLOW_ML_PER_S)  # مدة = جرعة ÷ تدفق 185
    pump_b.off()  # 186
    pump_a.on()  # المضخة A 187
    time.sleep(dose_a_ml / DOSE_FLOW_ML_PER_S)  # 188
    pump_a.off()  # 189
    print("DOSING done")  # 190
# <<< DOSING 191
```

6 · DOSING mode

Check the water, switch B off, run A for (dose ÷ flow) seconds, switch A off, run B, switch B off, print DOSING DONE. A and B are never on together. The volumes are only as accurate as the flow calibration of the real pumps (here 2 ml/s).

- Started by the DOSING button (or /mode/DOSING on the Pi web page).
- The Smart Farm counts the millilitres really pumped, so you can check the doses.

main.py

```
120  # >>> DOSING
(لا تعملان معاً أبداً) B ثم جرعة A جرعة #                                  :()def run_dosing  121
لا تسميد بدون ماء #                                 :()if not water_ok      122
نخبر بالسبب #         print("DOSING cancelled: LOW WATER")          123
نخرج من الدالة #                                          return          124
A متوقفة قبل تشغيل B تأكد أن #                                    ()dose_b.off      125
A تشغيل #                                       ()dose_a.on      126
A رسالة الجرعة #               print("DOSING A", DOSE_A_ML, "ml")      127
(ثوانٍ 5 = 2 ÷ 10) الزمن = الكمية ÷ التدفق #          time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)      128
A إيقاف #                                       ()dose_a.off      129
B ثم تشغيل #                                       ()dose_b.on      130
B رسالة الجرعة #               print("DOSING B", DOSE_B_ML, "ml")      131
ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)          # 5 ÷ 2 = 2.5      132
B إيقاف #                                       ()dose_b.off      133
انتهت الجرعات #                                print("DOSING DONE")      134
135  # <<< DOSING
```

7 · TEST mode

Each output runs for 2 s in turn: fan, light, dose A, dose B, then the pump — and the pump only if there is water. The protection stays active in test mode.

- Use it after wiring a real farm: every relay should click once and every load should run.
- Dose A and dose B run one after the other, never together.

main.py

```
تجربة كل جهاز لثانيتين #                                          :()def run_test  138
قائمة #  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]      139
↪ الأجهزة
اسم الجهاز الذي نختبره #                          print("TEST", name)          140
تشغيله #                                  ()device.on          141
ننتظر ثانيتين #                     time.sleep(TEST_SECONDS)          142
إيقافه #                                  ()device.off          143
(الحماية لا تُلغى) المضخة فقط إذا يوجد ماء #                                  :()if water_ok      144
نختبر المضخة #                               print("TEST PUMP")          145
تشغيلها #                                   ()pump.on          146
ننتظر ثانيتين #                     time.sleep(TEST_SECONDS)          147
إيقافها #                                   ()pump.off          148
لا يوجد ماء كافٍ #                                           :else      149
لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")          150
انتهى الاختبار #                               print("TEST DONE")      151
```

8 · Alarms

One if/elif chain picks the most important alarm: SENSOR ERROR, LOW WATER, PUMP FAILURE, HIGH TEMPERATURE, OUT OF RANGE, or NONE. The alarm lamp is on for any alarm, and the alarm is printed only when it changes.

- The order of the elif lines is the priority: a missing sensor matters more than a high temperature.
- PUMP FAILURE stays until the program restarts (the pump must be checked by a person).

main.py

```
179  # >>> ALARMS
اختيار أهم إنذار وتشغيل المصباح #                                  :()def check_alarms  180
سنغيّر هذا المتغير العام #                                global alarm      181
نحفظ الإنذار السابق #                                old = alarm      182
قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100      183
خطأ حساس #                     "alarm = "SENSOR ERROR          184
الماء قليل #                             :()elif not water_ok      185
نقص الماء #                         "alarm = "LOW WATER          186
المضخة لم تسحب تياراً #                                 :elif pump_failed      187
عطل المضخة #                       "alarm = "PUMP FAILURE          188
حرارة عالية #                     :elif temperature > MAX_TEMP      189
ارتفاع الحرارة #                   "alarm = "HIGH TEMPERATURE          190
خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX      191
قيمة خارج المجال #                      "alarm = "OUT OF RANGE          192
كل شيء طبيعي #                                   :else      193
لا يوجد إنذار #                               "alarm = "NONE          194
لا إنذار #                             :"if alarm == "NONE      195
```

```
إطفاء مصباح الإنذار #                              ()alarm_lamp.off          196
يوجد إنذار #                                             :else      197
تشغيل مصباح الإنذار #                              ()alarm_lamp.on          198
نطبع الإنذار عند تغيّره فقط #                                 :if alarm != old      199
رسالة الإنذار #                          print("ALARM", alarm)          200
201  # <<< ALARMS
```

9 · Display and monitoring

Every 5 s a STATUS line goes to the terminal, and the Flask page (Web tab) asks /status every 2 s. The main loop at the end is the whole program in 6 steps.

- STATUS mode=... temp=... ... alarm=... day=... is the same line on the three controllers.
- The status panel of this page reads that line from the running program.

main.py

```
سطر الحالة في الطرفية #                                    :()def show_status  210
211      print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
↪ الوضع والهواء
الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"            212
↪ والمضخة
213            "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
↪ الباقي
---------- (Flask) صفحة الويب (4 ---------- #  216
إنشاء تطبيق الويب #                                app = Flask(__name__)  217
218
219
الصفحة الرئيسية #                                          :()def home  220
templates/index.html الملف #            return render_template("index.html")      221
222
223
البيانات بصيغة JSON للصفحة #                                          :()def status  224
القراءات #  ,return jsonify(temperature=temperature, humidity=humidity, co2=co2, ph=ph, ec=ec      225
الحالة #  ,water_level=water_level, mode=mode, pump=on_off(pump), fan=on_off(fan)                    226
الإضاءة والإنذار واليوم #  (()light=on_off(light), alarm=alarm, day=growth_day                    227
228
229
زر في الصفحة يطلب TEST أو DOSING #                                    :def set_mode(name)  230
سنغيّر الوضع العام #                                      global mode      231
نقبل هذين الاسمين فقط #          :"if name == "DOSING" or name == "TEST      232
الحلقة الرئيسية تنفّذه #                                      mode = name          233
نرد بالوضع الحالي #                        return jsonify(mode=mode)      234
235
236
ربط العنوان / بالدالة home #                 app.add_url_rule("/", "home", home)  237
العنوان status/ يعطي القراءات #       app.add_url_rule("/status", "status", status)  238
العنوان mode/DOSING/ أو mode/TEST/ #  app.add_url_rule("/mode/<name>", "set_mode", set_mode)  239
الويب #  ()threading.Thread(target=app.run, kwargs={"host": "0.0.0.0", "port": 5000}, daemon=True).start  240
↪ في الخلفية
---------- الحلقة الرئيسية (5 ---------- #  242
رسالة البداية #        print("Smart farm started (Raspberry Pi 5)")  243
عدّاد الدورات #                                            loop = 0  244
تكرار إلى الأبد #                                         :while True  245
قراءة الحساسات (read_sensors()                                    # 1      246
زر الجرعات مضغوط؟ (if dosing_button.is_pressed:                      # 2      247
نختار وضع الجرعات #                                  "mode = "DOSING          248
زر الاختبار مضغوط؟ #                       :if test_button.is_pressed      249
نختار وضع الاختبار #                                    "mode = "TEST          250
تنفيذ الوضع (if mode == "DOSING":                                  # 3      251
الجرعات A ثم B #                                  ()run_dosing          252
ثم العودة إلى التلقائي #                                        "mode = "AUTO          253
وضع الاختبار #                                 :"elif mode == "TEST      254
تجربة الأجهزة #                                        ()run_test          255
ثم العودة إلى التلقائي #                                        "mode = "AUTO          256
الوضع AUTO #                                            :else      257
المروحة والإضاءة والري #                                    ()auto_control          258
الإنذارات (check_alarms()                                       # 4      259
الحالة كل 5 ثوانٍ (if loop % 5 == 0:                                      # 5      260
سطر STATUS #                                  ()show_status          261
الدورة التالية #                                        loop = loop + 1      262
انتظار ثانية ثم التكرار (time.sleep(1)                                          # 6      263
```

6. Complete program

main.py

```
# COMPETITION MASTER 01 — Raspberry Pi 5 Complete Smart Farm
Wiren Board و ESP32 نفس منطق مشروعي — Raspberry Pi 5 على مزرعة ذكية كاملة #
(sleep) الوقت والانتظار #                                        import time
فقط لتشغيل صفحة الويب في الخلفية #                                   import threading
CO2 لقراءة حساس UART منفذ #                                       import serial
DHT22 مكتبة حساس الحرارة والرطوبة #                                 import adafruit_dht
board.D16 أسماء أرجل اللوحة مثل #                                       import board
ADC محوّل + الأزرار + مخارج المرحّلات #  from gpiozero import OutputDevice, Button, MCP3008
صفحة الويب البسيطة #   from flask import Flask, jsonify, render_template

---------- الإعدادات: قيم تدريبية يمكنك تغييرها (1 ---------- #
فوق 30 درجة: تشغيل المروحة #                MAX_TEMP = 30
تحت 27 درجة: إيقاف المروحة #             FAN_OFF_TEMP = 27
أقل مستوى ماء مسموح لتشغيل المضخة (%) #                MIN_WATER = 20
مدة الري بالثواني #              PUMP_SECONDS = 5
ساعات الري في اليوم #  IRRIGATION_HOURS = [7, 12, 17]
ساعة تشغيل الإضاءة #             LIGHT_ON_HOUR = 6
ساعة إطفاء الإضاءة #           LIGHT_OFF_HOUR = 20
جرعة السماد A بالمليلتر #                DOSE_A_ML = 10
جرعة السماد B بالمليلتر #                DOSE_B_ML = 5
تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي) #           FLOW_ML_PER_SEC = 2
أقل pH مقبول #                  PH_MIN = 5.5
أعلى pH مقبول #                  PH_MAX = 6.5
أقل EC مقبول (mS/cm) #                  EC_MIN = 1.0
أعلى EC مقبول #                  EC_MAX = 2.5
أعلى CO2 مقبول (ppm) #               CO2_MAX = 1500
أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير) #           MIN_PUMP_AMPS = 0.2
مدة تشغيل كل جهاز في وضع الاختبار #              TEST_SECONDS = 2

# >>> PINS
---------- التوصيلات: كل جهاز على رجله (2 ---------- #
البيانات على GPIO16 :dht = adafruit_dht.DHT22(board.D16)                # DHT22
على UART0 (GPIO15 و GPIO14) co2_port = serial.Serial("/dev/serial0", 9600, timeout=1)  # MH-Z19B
حساس pH على قناة ADC رقم 0 #                         ph_adc = MCP3008(channel=0)
حساس EC على القناة 1 #                       ec_adc = MCP3008(channel=1)
حساس مستوى الماء على القناة 2 #                     level_adc = MCP3008(channel=2)
حساس تيار المضخة ACS712 على القناة 3 #                  current_adc = MCP3008(channel=3)
مرحّل مضخة الري على GPIO17 #                            pump = OutputDevice(17)
مرحّل المروحة على GPIO22 #                              fan = OutputDevice(22)
مرحّل الإضاءة على GPIO27 #                             light = OutputDevice(27)
مرحّل مصباح الإنذار على GPIO23 #                       alarm_lamp = OutputDevice(23)
مرحّل مضخة الجرعات A على GPIO5 #                           dose_a = OutputDevice(5)
مرحّل مضخة الجرعات B على GPIO6 #                           dose_b = OutputDevice(6)
زر وضع الجرعات على GPIO26 (إلى GND) #                       dosing_button = Button(26)
زر وضع الاختبار على GPIO19 (إلى GND) #                        test_button = Button(19)
# <<< PINS

---------- حالة النظام (3 ---------- #
الوضع الحالي: AUTO أو DOSING أو TEST #                "mode = "AUTO
الإنذار الحالي #               "alarm = "NONE
يصبح True إذا شغّلنا المضخة ولم يمر تيار #          pump_failed = False
آخر قراءة حرارة #           temperature = None
آخر قراءة رطوبة #              humidity = None
آخر قراءة CO2 #                 co2 = None
آخر قراءة pH #                    ph = 0
آخر قراءة EC #                    ec = 0
مستوى الماء % #              water_level = 0
تيار المضخة بالأمبير #               pump_amps = 0
آخر ساعة تم فيها الري (حتى لا نكرره) #     last_irrigation_hour = -1
وقت بدء التشغيل لحساب يوم النمو #      ()start_time = time.time
وقت آخر قراءة DHT22 (يحتاج ثانيتين بين القراءات) #             last_dht_time = 0


قراءة CO2 من MH-Z19B عبر UART #                                    :()def read_co2
أمر طلب القراءة (9 بايت) #  co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))
ننتظر رداً من 9 بايت #                        reply = co2_port.read(9)
لا يوجد رد = الحساس مفصول #                              :if len(reply) != 9
تعني: لا توجد قراءة return None                                  # None
البايت 2 و 3 = التركيز ppm #               return reply[2] * 256 + reply[3]


```

```
# >>> SENSORS
# قراءة كل الحساسات                                 :()def read_sensors
# سنغيّر هذه المتغيرات  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time
↪ العامة
if time.time() - last_dht_time >= 2:  قراءة جديدة كل ثانيتين فقط            # DHT22
# نحفظ وقت هذه القراءة                  ()last_dht_time = time.time
# نحاول القراءة                                      :try
# الحرارة °C           temperature = dht.temperature
# الرطوبة %                 humidity = dht.humidity
# فشل القراءة (الحساس مفصول)                        :except RuntimeError
# لا توجد قراءة حرارة                    temperature = None
# ولا رطوبة                           humidity = None
co2 = read_co2()  (أو None)                             # CO2 ppm
ph = round(ph_adc.voltage * 3.5, 2)  الجهد × 3.5          # pH = 3.5
ec = round(ec_adc.voltage / 0.5, 2)  الجهد ÷ 0.5          # EC
water_level = round(level_adc.voltage / 3.0 * 100)  # 3  فولت = %100
pump_amps = round((current_adc.voltage - 2.5) / 0.185, 2)  # ACS712: 2.5  فولت = صفر أمبير
# <<< SENSORS


# هل يوجد ماء كافٍ؟                                  :()def water_ok
# return water_level >= MIN_WATER and water_level <= 100  أكثر من %100 = قراءة غير صالحة


# يوم النمو منذ بدء التشغيل                                  :()def growth_day
return int((time.time() - start_time) / 86400) + 1  # 86400  ثانية = يوم واحد


# >>> IRRIGATION
# ري لمدة PUMP_SECONDS مع الحماية                                    :()def irrigate
# قد نغيّر هذا المتغير العام                              global pump_failed
print("IRRIGATION start")  رسالة بداية الري #
:for second in range(PUMP_SECONDS)  ثانية بعد ثانية #
()read_sensors  قراءة جديدة قبل كل ثانية #
:()if not water_ok  الماء نقص أثناء الري #
# print("IRRIGATION stopped: LOW WATER")  نخبر بالسبب
break  نخرج من الحلقة فوراً #
()pump.on  تشغيل المضخة #
time.sleep(1)  انتظار ثانية #
()read_sensors  نقرأ تيار المضخة الآن #
:if pump_amps < MIN_PUMP_AMPS  المضخة لا تسحب تياراً = لا تعمل #
pump_failed = True  نتذكر العطل (يظهر الإنذار) #
print("PUMP FAILURE: no current")  رسالة العطل #
break  نتوقف عن الري #
()pump.off  إيقاف المضخة دائماً في النهاية #
print("IRRIGATION end")  رسالة نهاية الري #
# <<< IRRIGATION


# >>> DOSING
# جرعة A ثم جرعة B (لا تعملان معاً أبداً)                                    :()def run_dosing
:()if not water_ok  لا تسميد بدون ماء #
print("DOSING cancelled: LOW WATER")  نخبر بالسبب #
return  نخرج من الدالة #
()dose_b.off  تأكد أن B متوقفة قبل تشغيل A #
()dose_a.on  تشغيل A #
print("DOSING A", DOSE_A_ML, "ml")  رسالة الجرعة A #
time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)  الزمن = الكمية ÷ التدفق (10 ÷ 2 = 5 ثوانٍ) #
()dose_a.off  إيقاف A #
()dose_b.on  ثم تشغيل B #
print("DOSING B", DOSE_B_ML, "ml")  رسالة الجرعة B #
time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)  # 5 ÷ 2 = 2.5  ثانية
()dose_b.off  إيقاف B #
print("DOSING DONE")  انتهت الجرعات #
# <<< DOSING


# تجربة كل جهاز لثانيتين                                    :()def run_test
# قائمة  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]
↪ الأجهزة
print("TEST", name)  اسم الجهاز الذي نختبره #
()device.on  تشغيله #
time.sleep(TEST_SECONDS)  ننتظر ثانيتين #
()device.off  إيقافه #
:()if water_ok  المضخة فقط إذا يوجد ماء (الحماية لا تُلغى) #
print("TEST PUMP")  نختبر المضخة #
()pump.on  تشغيلها #
```

```
147 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)
148 إيقافها #                                 ()pump.off
149 لا يوجد ماء كافٍ #                                    :else
150 لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")
151 انتهى الاختبار #                              print("TEST DONE")
152
153
154 الوضع التلقائي #                                 :()def auto_control
155 سنغيّر هذا المتغير العام #                    global last_irrigation_hour
156 (23 إلى 0) الساعة الحالية #                 hour = time.localtime().tm_hour
157     # >>> FAN
158 لا قراءة حرارة: نشغّل المروحة للأمان #                        :if temperature is None
159 تشغيل المروحة #                                    ()fan.on
160 حار جداً #                    :elif temperature > MAX_TEMP
161 تشغيل المروحة #                                    ()fan.on
162 برد بما يكفي #                :elif temperature < FAN_OFF_TEMP
163 (بين 27 و 30 تبقى كما هي) إيقاف المروحة #                                ()fan.off
164     # <<< FAN
165     # >>> LIGHT
166 داخل ساعات الإضاءة #  :if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR
167 تشغيل الإضاءة #                                   ()light.on
168 خارج ساعات الإضاءة #                                      :else
169 إطفاء الإضاءة #                                  ()light.off
170     # <<< LIGHT
171 حان وقت ري جديد #  :if hour in IRRIGATION_HOURS and hour != last_irrigation_hour
172 نتذكر الساعة حتى لا نكرر الري #                 last_irrigation_hour = hour
173 يوجد ماء؟ #                              :()if water_ok
174 نروي #                               ()irrigate
175 لا يوجد ماء #                                    :else
176 نتخطى الري #  print("IRRIGATION skipped: LOW WATER")
177
178
179 # >>> ALARMS
180 اختيار أهم إنذار وتشغيل المصباح #                                :()def check_alarms
181 سنغيّر هذا المتغير العام #                                   global alarm
182 نحفظ الإنذار السابق #                                    old = alarm
183 قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100
184 خطأ حساس #                       "alarm = "SENSOR ERROR
185 الماء قليل #                              :()elif not water_ok
186 نقص الماء #                           "alarm = "LOW WATER
187 المضخة لم تسحب تياراً #                                 :elif pump_failed
188 عطل المضخة #                         "alarm = "PUMP FAILURE
189 حرارة عالية #                       :elif temperature > MAX_TEMP
190 ارتفاع الحرارة #                    "alarm = "HIGH TEMPERATURE
191 خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX
192 قيمة خارج المجال #                        "alarm = "OUT OF RANGE
193 كل شيء طبيعي #                                    :else
194 لا يوجد إنذار #                                "alarm = "NONE
195 لا إنذار #                                :"if alarm == "NONE
196 إطفاء مصباح الإنذار #                                ()alarm_lamp.off
197 يوجد إنذار #                                       :else
198 تشغيل مصباح الإنذار #                                ()alarm_lamp.on
199 نطبع الإنذار عند تغيّره فقط #                                  :if alarm != old
200 رسالة الإنذار #                          print("ALARM", alarm)
201 # <<< ALARMS
202
203
204 لعرض حالة جهاز "OFF" أو "def on_off(device):                                    # "ON
205 الجهاز يعمل الآن؟ #                               :if device.is_active
206 يعمل #                                "return "ON
207 متوقف #                                    "return "OFF
208
209
210 سطر الحالة في الطرفية #                                    :()def show_status
211     print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
↪ الوضع والهواء
212 الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"
↪ والمضخة
213           "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
↪ الباقي
214
215
216 ---------- (Flask) صفحة الويب (4 ---------- #
217 إنشاء تطبيق الويب #                                 app = Flask(__name__)
218
219
220 الصفحة الرئيسية #                                          :()def home
```

```
templates/index.html الملف #            return render_template("index.html")


البيانات بصيغة JSON للصفحة #                                        :()def status
القراءات #  ,return jsonify(temperature=temperature, humidity=humidity, co2=co2, ph=ph, ec=ec
الحالة #  ,water_level=water_level, mode=mode, pump=on_off(pump), fan=on_off(fan)
الإضاءة والإنذار واليوم #  (()light=on_off(light), alarm=alarm, day=growth_day


زر في الصفحة يطلب DOSING أو TEST #                                  :def set_mode(name)
سنغيّر الوضع العام #                                    global mode
نقبل هذين الاسمين فقط #          :"if name == "DOSING" or name == "TEST
الحلقة الرئيسية تنفّذه #                                    mode = name
نرد بالوضع الحالي #                       return jsonify(mode=mode)


ربط العنوان / بالدالة home #                 app.add_url_rule("/", "home", home)
العنوان status/ يعطي القراءات #       app.add_url_rule("/status", "status", status)
العنوان mode/DOSING/ أو mode/TEST/ #  app.add_url_rule("/mode/<name>", "set_mode", set_mode)
الويب في الخلفية #  ()threading.Thread(target=app.run, kwargs={"host": "0.0.0.0", "port": 5000}, daemon=True).start


---------- الحلقة الرئيسية (5 ---------- #
رسالة البداية #        print("Smart farm started (Raspberry Pi 5)")
عدّاد الدورات #                                          loop = 0
تكرار إلى الأبد #                                       :while True
قراءة الحساسات (read_sensors()                                  # 1
زر الجرعات مضغوط؟ (if dosing_button.is_pressed:                    # 2
نختار وضع الجرعات #                            "mode = "DOSING
زر الاختبار مضغوط؟ #                       :if test_button.is_pressed
نختار وضع الاختبار #                             "mode = "TEST
تنفيذ الوضع (if mode == "DOSING":                                # 3
الجرعات A ثم B #                                ()run_dosing
ثم العودة إلى التلقائي #                               "mode = "AUTO
وضع الاختبار #                             :"elif mode == "TEST
تجربة الأجهزة #                                ()run_test
ثم العودة إلى التلقائي #                               "mode = "AUTO
الوضع AUTO #                                           :else
المروحة والإضاءة والري #                              ()auto_control
الإنذارات (check_alarms()                                  # 4
الحالة كل 5 ثوانٍ (if loop % 5 == 0:                                # 5
سطر STATUS #                                 ()show_status
الدورة التالية #                                      loop = loop + 1
انتظار ثانية ثم التكرار (time.sleep(1)                                  # 6

```

templates/index.html

```
<!doctype html>
<html lang="ar" dir="rtl"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,
initial-scale=1">
<title>Smart Farm — Raspberry Pi 5</title>
<style>
body{font-family:system-ui;margin:14px;background:#f2f6f3}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(120px,1fr));gap:8px}
.c{background:#fff;border-radius:10px;padding:8px;box-shadow:0 1px 3px #0002}.c b{display:block;font-size:22px}
button{font-size:16px;padding:8px 14px;margin:6px 4px}#alarm{font-weight:700;margin:10px 0}
</style></head>
<body>
<Raspberry Pi 5</h2 — المزرعة الذكية 🌱<h2>
<div class="grid">
<b id="humidity">--</b></div>% الرطوبة<"C<b id="temperature">--</b></div><div class="c° الحرارة<"div class="c>
 <div class="c">CO2 ppm<b id="co2">--</b></div><div class="c">pH<b id="ph">--</b></div>
<b id="water_level">--</b></div>% مستوى الماء<"div class="c">EC<b id="ec">--</b></div><div class="c>
<b id="day">--</b></div>يوم النمو<"b id="mode">--</b></div><div class="c>الوضع<"div class="c>
<b id="fan">--</b></div>المروحة<"b id="pump">--</b></div><div class="c>المضخة<"div class="c>
<b id="light">--</b></div>الإضاءة<"div class="c>
</div>
<div id="alarm">--</div>
<button/>تشغيل الجرعات<"button onclick="fetch('/mode/DOSING')>
<button/>وضع الاختبار<"button onclick="fetch('/mode/TEST')>
<script>
function update(){fetch('/status').then(function(r){return r.json()}).then(function(s){
 for (var k in s){var e=document.getElementById(k); if(e) e.textContent = s[k]===null ? '--' : s[k];}
{;({;s.alarm + ' :الإنذار' = document.getElementById('alarm').textContent
update(); setInterval(update, 2000);
</script></body></html>
```

```
29
```

7. Test scenarios and results

Results of the scenarios run in the browser on 2026-11-05 (real runtime of the simulator).

Scenario	What it does	Expected	Result
1 · Normal	08:00, 25 °C, water 75 %: run for 12 s without touching anything.	STATUS lines with all readings, mode AUTO, light ON (06–20 h), fan and pump OFF, alarm NONE, alarm lamp off.	PASS
2 · High temperature (25 → 35 °C)	At 3 s the air goes to 35 °C, at 9 s to 28.5 °C (between the two thresholds), at 15 s to 25 °C.	Fan ON and HIGH TEMPERATURE alarm above 30 °C; at 28.5 °C the alarm clears but the fan stays ON (hysteresis); below 27 °C the fan stops.	PASS
3 · Water low	11:59:55 — at 2 s the reservoir drops to 10 %; the 12:00 irrigation comes 3 s later.	LOW WATER alarm, alarm lamp ON, "IRRIGATION skipped: LOW WATER": the pump never runs.	PASS
4 · Irrigation at 12:00	Start at 11:59:56 and run 25 s.	"IRRIGATION start", the pump runs about 5 s (current measured every second), "IRRIGATION end", and only once in that hour.	PASS
5 · Dosing A then B	At 2 s press the DOSING button for 1.5 s.	DOSING A 10 ml (5 s), then DOSING B 5 ml (2.5 s), never together, then DOSING DONE and back to AUTO. Volumes come from the farm model of the pumps (2 ml/s).	PASS
6 · Test mode	21:00 (light off by schedule): at 2 s press the TEST button for 1.5 s.	TEST FAN, TEST LIGHT, TEST DOSE A, TEST DOSE B, TEST PUMP (only because the water is OK), TEST DONE; every load runs once, then AUTO switches the light off again.	PASS
7 · Sensor failure	At 3 s the air sensor (DHT22) gets the fault "Sensor disconnected"; at 10 s the fault is cleared.	SENSOR ERROR alarm, temp=None/null in STATUS, the fan runs (safe side) while the reading is missing; the alarm clears after repair.	PASS
8 · Lighting schedule	Start at 19:59:50 and run 18 s.	The grow light is ON before 20:00 and switches OFF after 20:00 (LIGHT_OFF_HOUR).	PASS
9 · Wiring error (pump wire loose)	The wire from relay contact NO1 to the ACS712 is removed, then the 12:00 irrigation starts (start 11:59:56).	The relay clicks but no current flows: "PUMP FAILURE: no current", PUMP FAILURE alarm, the relay is switched off at once.	PASS
10 · Full system	11:59:52: irrigation at 12:00, at 14 s the air reaches 35 °C, at 18 s press the DOSING button for 1.5 s, at 32 s the air is back to 25 °C.	Irrigation (≈5 s), fan + HIGH TEMPERATURE alarm, dosing A then B, then every alarm clears and the fan stops: the system returns to normal AUTO.	PASS

8. Change One Thing — exercises

1. Change the fan threshold

   Set MAX_TEMP to 28 and FAN_OFF_TEMP to 25. Run, then set the farm air temperature to 29 °C.

   You should see: The fan starts at 29 °C now and the HIGH TEMPERATURE alarm appears.

2. Change the irrigation time

   Set PUMP_SECONDS to 8 and IRRIGATION_HOURS to include the current hour. Run and watch the pump.

   You should see: IRRIGATION start … IRRIGATION end, about 8 s apart; the farm reservoir flow shows the pump running.

3. Move the fan to another GPIO

   Move the wire from GPIO22 (pin 15) to GPIO24 (pin 18) and change OutputDevice(22) to OutputDevice(24).

   You should see: The fan works again from the new pin. If only the code or only the wire changes, the fan stays off while the code thinks it is on (GPIO22).

4. Disconnect the level sensor

   While the program runs, delete the signal wire of the water-level sensor.

   You should see: The level reads wrong (near 0): LOW WATER alarm, the lamp lights, irrigation and dosing are refused. Put the wire back (Ctrl+Z).

5. Change the dose

   Set DOSE_A_ML to 20 and DOSE_B_ML to 10, then start DOSING.

   You should see: A runs 10 s and B 5 s; the Smart Farm totals show about 20 ml and 10 ml. Changing FLOW_ML_PER_SEC without changing the pump only changes the time, not the real flow — that is why calibration matters.

6. Break the pump while it runs

   Select the main pump, set its fault to "Pump failure" and start irrigation (put the current hour in IRRIGATION_HOURS).

   You should see: The relay closes but the ACS712 measures no current: PUMP FAILURE: no current, alarm PUMP FAILURE.

7. Change the lighting hours

   Set LIGHT_ON_HOUR to 8 and LIGHT_OFF_HOUR to 18. Use the clock (Fast-forward) to pass 18:00.

   You should see: The grow light switches off at 18:00 instead of 20:00.

8. Tighten the pH range

   Set PH_MAX to 6.0 (the farm solution is about pH 6.2).

   You should see: ALARM OUT OF RANGE and the lamp lights; nothing else changes because OUT OF RANGE only warns.

9. Building it on real hardware

1. Enable SPI, the serial port (disable the login shell on it) with sudo raspi-config → Interface Options.
2. Install the libraries (below) in a virtual environment: python3 -m venv --system-site-packages farm && . farm/bin/activate.
3. Wire with the Pi switched off. Relay modules: use the 5 V pin for VCC; the GPIO signal is 3.3 V — choose modules that switch at 3.3 V (active-HIGH as in the code).
4. The MH-Z19B needs 5 V power but its UART is 3.3 V: TX→GPIO15 (RX), RX←GPIO14 (TX).
5. pH/EC modules: calibrate with buffer solutions and replace the factors 3.5 and 0.5 with your calibration.
6. Copy main.py and templates/index.html, run python3 main.py, open http://<pi-address>:5000.

10. Libraries to install

- gpiozero (+ lgpio on Pi 5): sudo apt install python3-gpiozero python3-lgpio
- pyserial: sudo apt install python3-serial
- adafruit-circuitpython-dht: pip install adafruit-circuitpython-dht
- Flask: sudo apt install python3-flask
- MCP3008 (gpiozero, SPI): sudo raspi-config → Interface Options → SPI → enable

11. Simulation vs. real device

- The sensors, the ADC and the relays are simulated part models; their formulas are the simulator modules, not your real probes.
- The Flask page is served inside the simulator (Web tab); on the Pi it is a real server on port 5000.
- The DHT22 never fails randomly here; on real hardware an occasional read error is normal (the alarm can blink).
- Pump, fan and dosing pumps are resistive models; inrush current and back-EMF are not simulated — use relay modules with flyback protection.

These projects are training references; they do not prove which equipment the final uses nor cover every competition module.